Lab Instructions

In this lab, we focus on **simulating MPI-style distributed workflows** using Python multiprocessing—not installing real MPI. Students will implement core collective patterns such as scatter, gather, reduce, all-reduce, and broadcast, then apply them to a distributed word-count or numeric-statistics workflow.

MPI-style collective communication includes patterns such as broadcast, scatter, gather, allgather, reduce, allreduce, and barrier; scatter distributes different chunks from a root process, while gather collects results back to a root process. LLNL's HPC tutorial also describes collective operations as synchronization, data movement, and collective computation such as reductions. [github.com] [scientific....github.io]

Part A — Full Example Code

See July28_code.py

Part B — What Students Should Observe

Students should usually observe:

1. **Correctness should match the serial baseline.**
2. Increasing world_size may improve throughput at first.
3. Speedup may flatten or decrease because of:
    - process startup cost
    - data serialization
    - communication/gather overhead
    - reducer bottleneck
    - memory bandwidth
4. Word count may scale differently from numeric statistics.
5. Chunk sizes should be roughly balanced if using round-robin scatter.

This prepares students for Day 6 because distributed training uses similar communication ideas: **broadcast parameters**, compute local work, then **reduce/all-reduce updates**.

Part C — Student Results Table

Students should produce a table like this:

Task	Method	World Size	Time	Speedup	Throughput	Correct	Min Worker Time	Max Worker Time
word_count	serial	1		1.00		True		
word_count	distributed	2						
word_count	distributed	4						
numeric_stats	serial	1		1.00		True		
numeric_stats	distributed	4						

Part D — Discussion Questions

Ask students:

1. What is a **rank**?
2. What is **world size**?
3. What is the difference between:
    - broadcast and scatter?
    - gather and reduce?
    - reduce and all-reduce?
4. Why does every rank need to participate in a collective operation?
5. Why might a distributed version be slower than serial?
6. What data is communicated in:
    - scatter?
    - gather?

- reduce?

7. How does this pattern apply to:
    - image classification?
    - recommendation systems?
    - LLM request routing?
    - data-parallel training?

Part E —Implement broadcast_config

Students can simulate broadcasting model or experiment configuration.

Assuming you have the following configuration to broadcast, implement the simulation.

```
config = {
    "model_name": "tiny_classifier",
    "batch_size": 64,
    "seed": 42,
    "preprocess": "normalize",
}
```

Part F —Top-k Recommendation Reduce

Learn the following code and answer the questions at the end of this part.

```
import heapq
import random

def make_user_scores(num_users=100, num_items=1000):
    """
```

```
    Synthetic user-item candidate scores.
    """

    user_scores = {}

    for user_id in range(num_users):
        scores = []

        for item_id in range(num_items):
            score = random.random()
            scores.append((item_id, score))

        user_scores[user_id] = scores

    return user_scores



def local_top_k(user_score_pairs, k=5):
    """
    Compute local top-k item recommendations for a shard.
    """

    local_results = {}

    for user_id, scores in user_score_pairs:
        top_k = heapq.nlargest(
```

```
            k,
            scores,
            key=lambda x: x[1],
        )

        local_results[user_id] = top_k

    return local_results


def merge_top_k(partial_results, k=5):
    """
    Merge partial top-k recommendation results.
    """

    merged = {}

    for partial in partial_results:
        for user_id, top_items in partial.items():
            if user_id not in merged:
                merged[user_id] = []

            merged[user_id].extend(top_items)

    final = {}
```

```
    for user_id, candidates in merged.items():
        final[user_id] = heapq.nlargest(
            k,
            candidates,
            key=lambda x: x[1],
        )

    return final
```

Discussion:

Is top-k reduction the same as sum reduction?

What must each worker return?

Why should workers return compact top-k lists instead of all scores?

Part G —Simulate All-Reduce for Gradient Averaging

Learn the following code and answer the questions at the end of this part.

```
def average_vectors(vectors):
    """
    Average a list of numeric vectors.

    Example:
        vectors = [
            [1.0, 2.0],
            [3.0, 4.0],
        ]
```

```
    average = [2.0, 3.0]
    """

    n = len(vectors)
    length = len(vectors[0])

    avg = []

    for j in range(length):
        value = sum(v[j] for v in vectors) / n
        avg.append(value)

    return avg


def all_reduce_average(vectors):
    """
    Simulate all-reduce average.

    Every rank receives the averaged vector.
    """

    avg = average_vectors(vectors)

    return [
        avg
```

```
        for _ in vectors
    ]


local_gradients = [
    [1.0, 2.0, 3.0],
    [2.0, 3.0, 4.0],
    [3.0, 4.0, 5.0],
    [4.0, 5.0, 6.0],
]

averaged_gradients = all_reduce_average(local_gradients)

for rank, grad in enumerate(averaged_gradients):
    print(rank, grad)
```

Discussion:

Why does every rank need the same averaged gradient?

How is this different from gather-to-root?

Why is all-reduce important for data-parallel training?

Part H —Barrier Simulation

A simple barrier concept can be simulated using multiprocessing.Barrier. Learn the following code and answer the questions at the end of this part.

```
from multiprocessing import Process, Barrier
```

```
import time
import random


def barrier_worker(rank, barrier):
    print(f"rank {rank}: starting local work")
    time.sleep(random.uniform(0.5, 2.0))

    print(f"rank {rank}: waiting at barrier")
    barrier.wait()

    print(f"rank {rank}: passed barrier")


def run_barrier_demo(world_size=4):
    barrier = Barrier(world_size)
    processes = []

    for rank in range(world_size):
        p = Process(
            target=barrier_worker,
            args=(rank, barrier),
        )
        p.start()
        processes.append(p)
```

```
    for p in processes:
        p.join()


if __name__ == "__main__":
    run_barrier_demo()
```

Discussion:

Which rank arrived first?

Which rank arrived last?

Why did earlier ranks wait?

When are barriers useful?

When are they wasteful?

Part I — Deliverables

Each student submits a short report to my email, lidongvtemail@gmail.com

1. Results Table

Must include:

- task
- method
- world size
- runtime
- speedup
- throughput
- correctness
- worker timing

2. Short Reflection

Suggested prompt:

In 1–2 paragraphs:

1. What collectives did you implement?

2. How did you verify correctness?

3. Where did communication or aggregation overhead appear?

4. Did increasing world size always help?

5. How would your final project use scatter/gather/reduce/all-reduce